

1) allows ref struct generic constraint — use `Span<T>` / ref struct in generics

What: C# 13 introduces an *anti-constraint* allows ref struct so a generic type parameter can accept ref struct types (like `Span<T>` / `ReadOnlySpan<T>`). Previously, ref struct types could not be supplied as type arguments.

Why it matters: lets you write generic algorithms that operate over spans and other stack-only types without boxing or copying — big for performance and low-allocation code.

Example:

```
public class BufferProcessor<T> where T : allows ref struct
{
    public void Process(scoped T buffer) { /* ref-safety enforced by compiler */ }
}
```

Use-case: generic parsing/serialization over `Span<byte>` or `ReadOnlySpan<char>`.

2) ref struct types can implement interfaces (and related safety)

What: C# 13 allows ref struct types to implement interfaces. Conversions to interface types are still restricted (no boxing) — the compiler enforces ref-safety rules.

Why it matters: you can abstract over stack-only implementations using interfaces in generic code paths while preserving performance. That makes library APIs more composable while still permitting `Span<T>`-style zero-allocation patterns.

3) params collections for generic parameters

What: `params` is no longer limited to arrays — you can declare `params` for recognized collection types and interfaces (`IEnumerable<T>`, `ICollection<T>`, `ReadOnlySpan<T>`, etc.).

Why it matters: makes generic APIs that accept a variable number of items nicer to call and more efficient (e.g., `params ReadOnlySpan<T>` avoids allocations). It improves ergonomics of generic helper methods and overloads.

Example:

```
public void Concat<T>(params ReadOnlySpan<T> items) { /* iterate items */ }
```

Call-site becomes simpler and may avoid intermediate allocations.

4) Better method-group overload resolution & overload-priority (helps generic overloads)

What: Compiler improvements to method-group natural types and a new `OverloadResolutionPriorityAttribute` let the compiler pick better overloads (e.g., preferring a new

strongly-typed generic overload over older ones).

Why it matters: library authors can add improved generic overloads without surprising breaking changes; generics + overload resolution behave more predictably. Useful when designing generic API surfaces that evolve.

5) TensorPrimitives / Tensor<T> and generic numeric/vector operations (runtime & libraries)

What: .NET 9 adds TensorPrimitives and a generic Tensor<T> (experimental) with many SIMD-optimized generic overloads and numeric-style APIs. The primitives support generic overloads where T implements required interfaces.

Why it matters: enables high-performance, generic numeric and multi-dimensional data operations (AI/ML workloads) without duplicating code for each numeric type — leverages generic constraints and interface-based generic math patterns.

Example idea: a single generic algorithm over Tensor<T> that uses numeric APIs and benefits from SIMD and specialized overloads.

6) Static abstract members in interfaces → generic math & operator abstraction

What (background): Since C# 11, interfaces may declare static abstract members (the language feature that enabled .NET Generic Math). That lets you require numeric operators (like +, /) at the interface level so generic code can call them. This capability is central to doing arithmetic over T in a generic method (e.g., where T : INumber<T>).

Why it matters for .NET 9: .NET 9 and C# 13 continue and expand the ecosystem (libraries, primitives, and runtime support) around generic math; the static-abstract pattern remains the canonical way to express numeric operator requirements for generics.

Example:

```
static T Middle<T>(T a, T b) where T : System.Numerics.INumber<T>
```

```
=> (a + b) / T.CreateChecked(2);
```

This compiles because INumber<T> provides the static abstract operators.

Short practical takeaways

- **Performance-first generics:** allows ref struct + ref struct interfaces let you write *generic*, *zero-allocation* algorithms (work with `Span<T>` safely).
- **Generic numeric code is more powerful:** static-abstract interface members + numeric interfaces (`INumber`, etc.) + .NET 9 numeric/tensor primitives enable writing *single* generic algorithms that work across numeric types and are SIMD-friendly.
- **API ergonomics improved:** params collections and overload resolution improvements make generic APIs easier to author and call.